

## Section 2.2: Gradient Tree Boosting

微積が苦手な方向けの、式の導出と直感的解説

### 前提：ブースティングとは

XGBoostは**ブースティング**という考え方に基づいています。「弱い予測モデルを少しずつ積み重ねて、精度を上げていく」アプローチです。

#### 💡 直感

たとえば「テストの点数を予測する」場面を考えます。

最初のモデルが 60 点と予測 → 実際は 75 点 → **15 点の誤差**。

次のモデルはその 15 点の誤差を補正しようとしします。さらに次のモデルが残りを補正... これを繰り返します。Section 2.2 は「次に加える木をどう賢く選ぶか」の数学的仕組みです。

### ステップ 1：目標を式にする

$t$  回目のステップで、新しい木  $f_t$  を加えて最小化したい損失は次のとおりです：

目標関数（ $T$ ステップ目）

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

| 記号                  | 意味                                  |
|---------------------|-------------------------------------|
| $l(y_i, \hat{y}_i)$ | 予測値 $\hat{y}_i$ と正解 $y_i$ のずれ（損失関数） |
| $\hat{y}_i^{(t-1)}$ | 前のステップまでの累積予測値                      |
| $f_t(x_i)$          | 今回新たに加える木の予測値                       |
| $\Omega(f_t)$       | 木が複雑になりすぎることへのペナルティ（正則化）            |

「今までの予測 + 新しい木」で誤差を最小にしたい、ということを表した式です。

### ステップ 2：テイラー展開で近似する

この式をそのまま最小化するのは一般に難しいため、**テイラー展開**による二次近似を使って扱いやすい形に変えます。

#### テイラー展開の直感

## 💡 直感

山道を歩いているとして、「少し先の高さ」を予測したいとします。

一次近似：今いる地点の「傾き」だけ見て予測する。

二次近似：「傾き」＋「カーブのきつさ」まで見て予測する → より正確。

XGBoost は二次近似を採用し、精度と計算効率のバランスを取っています。

損失関数  $l$  を  $\hat{y}_i^{(t-1)}$  の周りで二次展開すると：

### 二次テイラー近似

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[ l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

| 記号    | 数学的意味                                                                               | 直感的イメージ                       |
|-------|-------------------------------------------------------------------------------------|-------------------------------|
| $g_i$ | 損失の一階微分 $\frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$     | 今の予測が正解に対して「どちら向きにずれているか」（傾き） |
| $h_i$ | 損失の二階微分 $\frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)^2}}$ | そのずれの「カーブのきつさ」（曲率）            |

第一項  $l(y_i, \hat{y}_i^{(t-1)})$  は前のステップから変わらない定数なので、最小化に関係しません。これを除いた簡略化された目標関数が：

### 簡略化後の目標関数（式 3）

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[ g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t) \quad (3)$$

## 📌 ポイント

この式のポイントは、**損失関数の形によらず**  $g_i$  と  $h_i$  を計算さえすれば同じ枠組みが使えることです。回帰・分類・ランキングなど、どんなタスクにも対応できる汎用性がここに宿っています。

## ステップ 3：木の構造に合わせて整理する

正則化項を展開し、木の「葉」ごとにまとめ直します。葉  $j$  に落ちるデータの集合を  $I_j = \{i \mid q(x_i) = j\}$ 、葉のスコア（重み）を  $w_j$  とすると：

### 葉ごとに整理した目標関数（式 4）

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[ \left( \sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left( \sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T \quad (4)$$

各葉について、 $w_j$  に関する二次式 ( $aw_j^2 + bw_j$  の形) になっています。二次式は下に凸の放物線なので、頂点で最小値を取ります。

## ステップ 4：最適な葉のスコアを求める

式 (4) を  $w_j$  について微分してゼロと置くと、各葉の**最適スコア**が解析的に求まります：

最適スコア (式 5)

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (5)$$

部分

意味

分子  $\sum_{i \in I_j} g_i$

その葉に入ったデータの「誤差の向き」の合計

分母  $\sum_{i \in I_j} h_i + \lambda$

「曲率の合計」+ 正則化パラメータ  $\lambda$

この最適スコアを式 (4) に代入すると、**木の構造  $q$  の良さを測るスコア**が得られます：

構造スコア (式 6)

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{\left( \sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \quad (6)$$

🔴 ポイント

このスコアが**小さいほど良い木**です（マイナスが大きい = 損失が小さい）。決定木の不純度指標（ジニ係数など）に相当しますが、こちらはどんな損失関数にも対応できます。

## ステップ 5：分割の価値を評価する

すべての木の構造を列挙するのは現実的でないため、**貪欲法 (greedy)** で「今ある葉をどこで分けるか」を一つずつ決めていきます。その判断基準が  $L_{\text{split}}$  です。

式 (6) からの導出

ある葉を左 ( $I_L$ ) と右 ( $I_R$ ) に分割する前後を、式 (6) で比較します：

状況

式 (6) の当該葉の寄与

葉の数

分割前 (葉 1 枚)

$$-\frac{1}{2} \cdot \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}$$

$T$

| 状況          | 式 (6) の当該葉の寄与                                                                                    | 葉の数     |
|-------------|--------------------------------------------------------------------------------------------------|---------|
| 分割後 (葉 2 枚) | $-\frac{1}{2} \cdot \frac{G_L^2}{H_L + \lambda} - \frac{1}{2} \cdot \frac{G_R^2}{H_R + \lambda}$ | $T + 1$ |

「分割後の損失」から「分割前の損失」を引いた差 (= 損失の減少量) が  $L_{\text{split}}$  です。葉が 1 枚増えるので、 $\gamma T$  の差分として  $\gamma$  が引かれます：

分割利得 (式 7)

$$L_{\text{split}} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (7)$$

| 記号                           | 意味                   |
|------------------------------|----------------------|
| $G_L = \sum_{i \in I_L} g_i$ | 左の葉に入るデータの $g_i$ の合計 |
| $G_R = \sum_{i \in I_R} g_i$ | 右の葉に入るデータの $g_i$ の合計 |
| $H_L = \sum_{i \in I_L} h_i$ | 左の葉に入るデータの $h_i$ の合計 |
| $H_R = \sum_{i \in I_R} h_i$ | 右の葉に入るデータの $h_i$ の合計 |
| $\gamma$                     | 葉を 1 枚増やすことへのペナルティ   |

💡 直感

角括弧の中は「分割によって得られる利得」、 $\gamma$  は「葉を 1 枚増やすコスト」です。

$L_{\text{split}} > 0$  のときだけ分割する価値がある、ということがこの式から直接読み取れます。

$\gamma$  を大きくするほど分割が抑制され、木が浅く (シンプルに) なります。

## Section 2.2 の流れまとめ

① 目標関数を立てる : 新しい木  $f_t$  を加えて損失を最小化したい



② テイラー展開で近似 :  $g_i$  (傾き) と  $h_i$  (曲率) を使って扱いやすい二次式にする



③ 葉ごとに整理 : 木の構造に合わせて式を書き換える



④ 最適スコアを解析的に計算 : 微分 = 0 で  $w_j^*$  が一発で求まる



⑤ 木の「良さスコア」を定義 :  $w_j^*$  を代入した式 (6) が構造評価の基準



⑥ 分割前後の差分で  $L_{\text{split}}$  を導出 : 貪欲法で最良の分割点を探索

## XGBoost がこの枠組みで強い理由

1. **汎用性** : 損失関数を替えるだけで、回帰・分類・ランキングなど何にでも使える
2. **正則化が組み込まれている** :  $\lambda$  (スコアへのペナルティ) と  $\gamma$  (葉の数へのペナルティ) で過学習を自然に防ぐ
3. **計算効率** : 最適スコアが解析的に求まるため、試行錯誤なしに一発で計算できる

---

出典 : Tianqi Chen & Carlos Guestrin, "XGBoost: A Scalable Tree Boosting System", KDD 2016. arXiv:1603.02754